

AI-Powered PDF to E-Course Learning Platform

Full Build Plan — Next.js + FastAPI + Neon PostgreSQL + Groq

Contents

1. Tech Stack Summary	3
2. Recommended Extra Features	4
3. High-Level Architecture	5
4. Authentication — Google OAuth + Email/Password	6
4.1 Approach	6
4.2 Database Schema	6
4.3 Google OAuth Setup (Google Cloud Console)	7
4.4 Next.js — Auth.js Configuration	7
4.5 Email/Password Registration Flow	8
4.6 Password Reset Flow	9
4.7 FastAPI — Verifying the Session	9
4.8 Security Checklist for Auth	10
5. PDF Upload and Processing	11
6. AI Course Generation	12
7. Learning Progress Tracking	13
8. AI Learning Companion Chatbot (RAG-based)	14
9. Quiz Generation	15
10. Course Dashboard	16
11. Search	17
12. Learning History	18
13. Responsive UI	19
14. Database Schema (Full, Simplified)	20
15. Core API Endpoints (FastAPI)	21

16. Build Order (Phased Plan)	22
17. Notes on Groq and Free-Tier Constraints	23
18. What Will Score Well in Evaluation	24

1. Tech Stack Summary

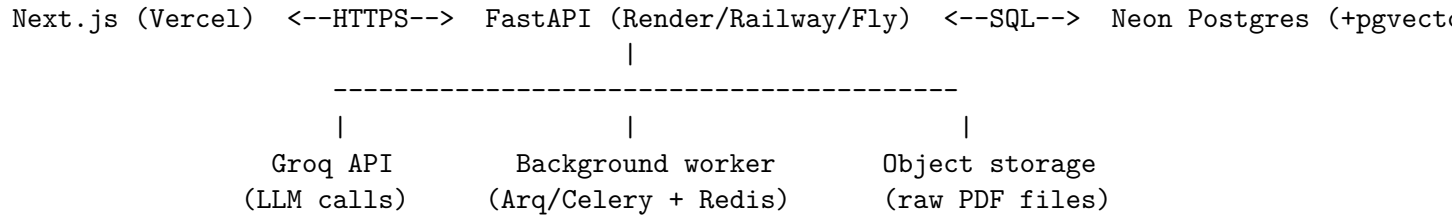
Layer	Choice
Frontend	Next.js (App Router) + Tailwind CSS + shadcn/ui
Backend	FastAPI (async)
Database	Neon PostgreSQL (+ pgvector extension)
LLM Inference	Groq API
Auth	Auth.js (NextAuth v5) — Google OAuth + Email/Password
Background Jobs	Arq or Celery (Redis-backed)
File Storage	Supabase Storage / Cloudflare R2 / S3 (for raw PDFs)
Deployment	Vercel (frontend), Render/Railway/Fly.io (backend + worker)

2. Recommended Extra Features

These go beyond the base assignment brief and are chosen specifically because they are cheap to build on your stack (Next.js + FastAPI + Neon + Groq) while meaningfully improving the evaluation score.

1. **RAG with pgvector** — Neon supports the `pgvector` extension natively, so retrieval-augmented generation for both course content and the chatbot can be built without standing up a separate vector database (ChromaDB/Pinecone/FAISS). This single decision satisfies several “bonus” requirements at once (RAG, semantic search, vector DB).
2. **Streaming responses (SSE)** — Groq’s inference is very fast (LPU-based), so streaming both course generation progress and chatbot replies token-by-token makes the app feel dramatically more responsive than waiting for a full response.
3. **Flashcards with spaced repetition** — auto-generate flashcards from lesson content and schedule reviews with a simple SM-2 algorithm.
4. **Mind maps per chapter** — ask the LLM for a JSON tree of concepts and render it with `react-flow` or `d3`. Visually strong for a live demo.
5. **Audio narration (TTS)** — use a free-tier TTS provider, or fall back to the browser’s built-in `SpeechSynthesis` API for zero cost.
6. **Auto-generated course certificate (PDF)** — generate a downloadable certificate when a user reaches 100% course completion.
7. **Multi-PDF course merge** — allow a course to be built from more than one uploaded PDF (e.g. a syllabus split into several files).
8. **“Explain like I’m 5” / “Expert mode” toggle** — same underlying content, different system prompt depth. Cheap to add and demonstrates prompt engineering skill directly.
9. **Adaptive quiz difficulty** — track a user’s quiz performance over time and adjust the difficulty of subsequently generated questions.
10. **Learning streaks + XP/levels** — simple gamification layer on top of existing progress data.
11. **Public/private course sharing links** — share a read-only version of a generated course.
12. **Export course as Markdown or PDF** — reuse the PDF-generation pipeline in reverse.
13. **Dark mode** — expected in any modern web app, trivial with Tailwind + `shadcn`.
14. **Rate limiting / usage quotas per user** — important given Groq’s free-tier request limits; also demonstrates production-readiness.
15. **Background job queue for PDF processing** — large PDFs should never block the HTTP request; processing happens asynchronously with live progress updates.

3. High-Level Architecture



Key architectural decisions:

- **Async everywhere in FastAPI** (async def route handlers, `asyncpg` or SQLAlchemy's async engine) because PDF processing and LLM calls are I/O-bound and benefit heavily from concurrency.
- **Background jobs** handle PDF parsing and course generation, since these can take 30 seconds to several minutes for large documents. Arq is a good fit — lightweight, Redis-based, and async-native, pairing naturally with FastAPI. Celery is a heavier but more feature-rich alternative.
- **Server-Sent Events (SSE)** are used for one-directional streaming: course-generation progress updates and chatbot token streaming. SSE is simpler than WebSockets for this use case and works reliably through most free-tier hosting providers.
- **pgvector inside Neon** stores chunk embeddings for retrieval-augmented generation, avoiding the operational overhead of a separate vector database service.

4. Authentication — Google OAuth + Email/Password

4.1 Approach

Since both **Google OAuth** and a **self-hosted email/password system** are required, **Auth.js** (**NextAuth v5**) is the right tool for the Next.js side. Unlike an all-in-one provider such as Clerk (which owns the user table on its own servers), Auth.js lets the user table live in your own Neon database, alongside all other app data — which fits naturally with the rest of this project’s schema.

FastAPI’s role is simply to **verify the session/JWT issued by Auth.js** on each incoming request — it does not need to implement its own OAuth flow.

4.2 Database Schema

```
CREATE EXTENSION IF NOT EXISTS pgcrypto; -- for gen_random_uuid()

CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email TEXT UNIQUE NOT NULL,
  name TEXT,
  avatar_url TEXT,
  email_verified TIMESTAMPTZ,
  password_hash TEXT,          -- NULL if the user has only ever used Google
  created_at TIMESTAMPTZ DEFAULT now()
);

-- Links each login method to a user (supports one user having both
-- a Google login AND a password login on the same account)
CREATE TABLE accounts (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  provider TEXT NOT NULL,      -- 'google' / 'credentials'
  provider_account_id TEXT,    -- Google's unique "sub" identifier
  UNIQUE(provider, provider_account_id)
);

CREATE TABLE password_reset_tokens (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  token_hash TEXT NOT NULL,
  expires_at TIMESTAMPTZ NOT NULL,
  used BOOLEAN DEFAULT false
);

CREATE TABLE email_verification_tokens (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  token_hash TEXT NOT NULL,
```

```
    expires_at TIMESTAMPTZ NOT NULL
  );
```

A user can have `password_hash = NULL` if they signed up through Google only, and can later choose to “add a password” from account settings. Likewise, a user who registers with email/password can later link a Google account. Account-linking by matching email address should only happen automatically if `email_verified` is set, to prevent account-takeover via an unverified email claim.

4.3 Google OAuth Setup (Google Cloud Console)

1. Create a project in Google Cloud Console.
2. Configure the OAuth consent screen (external, add your app name/logo/support email).
3. Create OAuth 2.0 credentials of type “Web application”.
4. Add authorized redirect URIs:
 - `http://localhost:3000/api/auth/callback/google` (development)
 - `https://your-domain.vercel.app/api/auth/callback/google` (production)
5. Copy the generated `Client ID` and `Client Secret` into environment variables:
 - `GOOGLE_CLIENT_ID`
 - `GOOGLE_CLIENT_SECRET`

4.4 Next.js — Auth.js Configuration

```
// auth.ts
import NextAuth from "next-auth";
import Google from "next-auth/providers/google";
import Credentials from "next-auth/providers/credentials";
import bcrypt from "bcryptjs";
import { db } from "@/lib/db";

export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [
    Google({
      clientId: process.env.GOOGLE_CLIENT_ID!,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET!,
    }),
    Credentials({
      credentials: { email: {}, password: {} },
      authorize: async (creds) => {
        const user = await db.query.users.findFirst({
          where: (u, { eq }) => eq(u.email, creds.email as string),
        });
        if (!user || !user.password_hash) return null;

        const valid = await bcrypt.compare(
          creds.password as string,
          user.password_hash
        );
        if (!valid) return null;
      },
    }),
  ],
});
```

```

        if (!user.email_verified) throw new Error("EMAIL_NOT_VERIFIED");

        return { id: user.id, email: user.email, name: user.name };
    },
    }),
],
session: { strategy: "jwt" },
callbacks: {
    async signIn({ user, account }) {
        if (account?.provider === "google") {
            await upsertGoogleUser(user); // create or link by verified email
        }
        return true;
    },
    async jwt({ token, user }) {
        if (user) token.userId = user.id;
        return token;
    },
    async session({ session, token }) {
        session.userId = token.userId as string;
        return session;
    },
},
pages: { signIn: "/login" },
});

```

```

// app/api/auth/[...nextauth]/route.ts
export { handlers as GET, handlers as POST } from "@auth";

```

4.5 Email/Password Registration Flow

Registration is handled through a **separate custom route**, since Auth.js's Credentials provider is designed for login, not account creation.

```

// app/api/auth/register/route.ts
export async function POST(req: Request) {
    const { email, password, name } = await req.json();

    const existing = await db.query.users.findFirst({
        where: (u, { eq }) => eq(u.email, email),
    });
    if (existing) {
        return Response.json({ error: "Email already registered" }, { status: 409 });
    }

    const password_hash = await bcrypt.hash(password, 12);
    const [newUser] = await db.insert(users)
        .values({ email, name, password_hash })

```



```

        .returning();

    await sendVerificationEmail(newUser); // creates a token + emails a link

    return Response.json({ success: true });
}

```

Steps in the full flow:

1. User submits the signup form (name, email, password).
2. Backend checks for an existing account with that email.
3. Password is hashed with **bcrypt** (never stored in plain text; cost factor 12 is a reasonable default).
4. A new **users** row is created with **email_verified** = NULL.
5. An email verification token is generated (random string, stored **hashed**, with an expiry — e.g. 24 hours) and emailed to the user as a clickable link.
6. On visiting the verification link, the backend checks the token, sets **email_verified** = **now()**, and marks the token as used.
7. Login via Credentials provider is blocked until **email_verified** is set (see **authorize** above), preventing unverified accounts from accessing the app.

4.6 Password Reset Flow

1. User requests a reset via “Forgot password?” and submits their email.
2. Backend generates a random token, stores its **hash** (never the raw token) in **password_reset_tokens** with a short expiry (e.g. 1 hour), and emails a reset link containing the raw token.
3. User opens the link and submits a new password.
4. Backend verifies the token against the stored hash, checks it hasn’t expired or already been used, hashes the new password with **bcrypt**, updates **users.password_hash**, and marks the token **used** = **true**.
5. All existing sessions for that user should be invalidated after a successful reset (rotate the JWT signing context or maintain a **session_version** column checked on every request).

4.7 FastAPI — Verifying the Session

FastAPI does not issue its own tokens; it verifies the JWT that Auth.js issues to the browser.

```

# auth.py
from fastapi import Depends, HTTPException, Request
from jose import jwt, JWTError

NEXTAUTH_SECRET = settings.NEXTAUTH_SECRET # shared secret with Next.js

async def get_current_user(request: Request) -> dict:
    token = request.cookies.get("authjs.session-token") or \
        request.headers.get("Authorization", "").removeprefix("Bearer ")
    if not token:
        raise HTTPException(status_code=401, detail="Not authenticated")
    try:

```

```

        payload = jwt.decode(token, NEXTAUTH_SECRET, algorithms=["HS256"])
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid or expired session")

    user_id = payload.get("userId")
    if not user_id:
        raise HTTPException(status_code=401, detail="Malformed session")
    return {"id": user_id}

```

Every protected FastAPI route depends on `get_current_user`:

```

@app.get("/dashboard")
async def dashboard(user: dict = Depends(get_current_user)):
    ...

```

4.8 Security Checklist for Auth

- Passwords hashed with **bcrypt** (or **argon2**), never stored or logged in plain text.
- Email verification required before password-based login is allowed.
- Password reset and email verification tokens are **random, hashed at rest, single-use, and short-lived**.
- Rate-limit login and registration endpoints to prevent brute-force and credential-stuffing attacks (e.g. 5 attempts per 15 minutes per IP/email).
- Account-linking by email between Google and Credentials providers only happens when the email is verified, to prevent account takeover.
- Sessions use HTTP-only, secure cookies; the shared `NEXTAUTH_SECRET` between Next.js and FastAPI must be a strong random value and never exposed to the client.
- CSRF protection is handled by Auth.js by default for its own routes — for custom routes like `/api/auth/register`, apply standard CSRF/CORS protections.

5. PDF Upload and Processing

- **Frontend:** drag-and-drop uploader (e.g. `react-dropzone`) with a visible upload progress bar.
- **Storage:** the raw PDF is uploaded to object storage (Supabase Storage, Cloudflare R2, or AWS S3's free tier) rather than stored inside Postgres; only the resulting URL/key is saved in the database.
- **Backend flow:**
 1. `POST /documents/upload` saves the file, creates a `document` row with status `processing`, enqueues a background job, and returns the `document_id` immediately (the request does not block).
 2. The background worker then:
 - Extracts text using `PyMuPDF` (fast, handles large PDFs well), with `pdfplumber` as a fallback for documents with unusual layouts.
 - Chunks the extracted text (roughly 500–1000 tokens per chunk, with overlap) to prepare it for retrieval-augmented generation.
 - Generates embeddings for each chunk — using a free, locally-run embedding model (e.g. via `sentence-transformers`) avoids needing a paid embeddings API — and stores them in a `document_chunks` table with a `vector` column.
 - Updates `document.status` to `ready` once processing completes.
 3. The frontend polls or subscribes via SSE to reflect live status transitions: `uploading` → `processing` → `generating_course` → `ready`.
- **Handling large PDFs:** stream-parse the document page by page rather than loading it entirely into memory, and enforce a maximum page count or file size with a clear error message if the document exceeds it.

6. AI Course Generation

Course generation uses a **two-stage prompting strategy** to balance output quality against latency and cost:

1. **Outline pass** — the document’s chunks/summary are sent to Groq to produce course meta-data (title, description, estimated learning time, objectives, prerequisites, difficulty level) plus a full table of contents (chapters → topics → subtopics), returned as **structured JSON**.
2. **Lesson pass** — for each topic/subtopic, the actual lesson content is generated (explanation, key takeaways, important notes, real-world examples, summary). Relevant source chunks are retrieved via RAG for each lesson so the content stays grounded in the original PDF rather than hallucinating.

Additional implementation notes:

- Lesson generation runs in **parallel batches** (e.g. `asyncio.gather` with a concurrency limit) to reduce total course-generation time, while respecting Groq’s rate limits.
- Progress is streamed to the frontend over SSE (e.g. “Generating chapter 2 of 6...”).
- All generated content is persisted relationally: `courses` → `chapters` → `topics` → `lessons`.

Prompt engineering practices to apply and highlight in documentation:

- A fixed system prompt establishes tone, depth, and required output format.
- JSON output is enforced via an explicit schema in the prompt (and `response_format` if the chosen Groq model supports it), with a JSON-repair fallback parser for resilience.
- Few-shot examples of a “good lesson” are included to anchor structure and quality.

7. Learning Progress Tracking

- A `progress` table records: `user_id`, `lesson_id`, `status` (`not_started`/`in_progress`/`completed`), `completed_at`, `time_spent_seconds`.
- Chapter and course completion percentages can be computed on read, or maintained as a denormalized `courses.completion_pct` field updated on write for faster dashboard queries.
- “Resume learning” is implemented via an `enrollments` table (`user_id`, `course_id`, `last_lesson_id`, `last_accessed_at`) tracking the most recently accessed lesson per course.

8. AI Learning Companion Chatbot (RAG-based)

- Endpoint: POST /chat with body {course_id, message, conversation_id}.
- **Flow:**
 1. Embed the user's incoming message.
 2. Perform a vector search over `document_chunks` (pgvector cosine similarity: `ORDER BY embedding <=> query_embedding LIMIT k`), scoped to the course's source document.
 3. Construct a prompt combining system instructions, the retrieved chunks, recent chat history (last N turns), and the user's message.
 4. Stream Groq's response back to the client over SSE.
- Full chat history is stored in `chat_messages` (role, content, course_id, created_at), satisfying the "Learning History" requirement.
- Special chatbot capabilities — quiz-on-demand, chapter summarization, "suggest next lesson" — are implemented as **intent-routed prompt templates**: detected either via explicit UI buttons ("Summarize this chapter", "Quiz me") or lightweight intent classification, each routing to its own dedicated prompt rather than a single generic chat prompt.

9. Quiz Generation

- Endpoint: `POST /courses/{id}/chapters/{chapter_id}/quiz` generates a configurable number of questions (a mix of multiple-choice, true/false, and short-answer), grounded in that chapter's lesson content via RAG.
- Storage: structured JSON persisted in `quizzes` and `quiz_questions` tables (question text, options, correct answer, explanation, question type).
- **Grading:**
 - Multiple-choice and true/false: graded instantly via exact match.
 - Short-answer: the user's answer and the correct answer are sent to Groq for semantic grading, returning a score (0–1) plus brief feedback.
- Attempts are stored in `quiz_attempts` (score, answers as JSON, submitted_at), supporting retakes and the adaptive-difficulty bonus feature.

10. Course Dashboard

- A single `GET /dashboard` endpoint aggregates: enrolled courses with completion percentages, recently accessed courses, total time spent (summed from `progress`), average quiz scores, and learning streak.
- The frontend renders this as a card/grid layout, with charts (e.g. via `recharts`) for visualizing progress over time.

11. Search

Search is implemented in two layers:

1. **Keyword search** — Postgres full-text search (`tsvector/tsquery`) across chapter, topic, and lesson titles and content. Fast, and requires no additional infrastructure.
2. **Semantic search (bonus)** — reuses the same pgvector chunk embeddings to let users search by concept rather than exact keyword match.

Endpoint: `GET /search?q=...&course_id=...`, returning ranked results with deep links directly into the relevant lesson.

12. Learning History

Since documents, courses, progress, chat messages, and quiz attempts are already relationally tied to `user_id`, learning history is primarily an **aggregation layer** rather than new storage:

- `GET /history` returns a merged timeline of uploads, lesson completions, quiz attempts, and chat sessions, sorted chronologically.

13. Responsive UI

- Built with Next.js App Router, Tailwind CSS, and shadcn/ui components.
- Key screens: Landing → Auth (login/signup, Google + email/password) → Dashboard → Upload → Course Generation (live progress) → Course Viewer (sidebar table of contents + lesson pane + chat drawer) → Quiz → Profile/History.
- Mobile adaptations: the sidebar collapses into a drawer, and the chatbot appears as a bottom sheet rather than a side panel.

14. Database Schema (Full, Simplified)

```
users(id, email, name, avatar_url, email_verified, password_hash, created_at)
accounts(id, user_id, provider, provider_account_id)
password_reset_tokens(id, user_id, token_hash, expires_at, used)
email_verification_tokens(id, user_id, token_hash, expires_at)

documents(id, user_id, filename, storage_url, page_count, status, uploaded_at)
document_chunks(id, document_id, chunk_index, content, embedding vector(384))

courses(id, document_id, user_id, title, description, difficulty,
         est_learning_time, objectives text[], prerequisites text[], created_at)

chapters(id, course_id, title, order_index)
topics(id, chapter_id, title, order_index)
lessons(id, topic_id, title, content_md, key_takeaways, notes, examples,
        summary, order_index)

enrollments(id, user_id, course_id, last_lesson_id, last_accessed_at)
progress(id, user_id, lesson_id, status, completed_at, time_spent_seconds)

chat_conversations(id, user_id, course_id, created_at)
chat_messages(id, conversation_id, role, content, created_at)

quizzes(id, chapter_id, generated_at)
quiz_questions(id, quiz_id, type, question, options jsonb, correct_answer,
              explanation)
quiz_attempts(id, quiz_id, user_id, answers jsonb, score, submitted_at)

flashcards(id, lesson_id, front, back, ease_factor, interval_days,
           next_review_at)           -- bonus
streaks(user_id, current_streak, longest_streak, last_active_date) -- bonus
```

Enable pgvector once per database: `CREATE EXTENSION IF NOT EXISTS vector;`

15. Core API Endpoints (FastAPI)

POST	/auth/register	-- email/password signup
POST	/auth/verify-email	-- confirm email verification token
POST	/auth/forgot-password	-- request password reset
POST	/auth/reset-password	-- submit new password with token
POST	/documents/upload	-- upload PDF, kick off processing
GET	/documents/{id}/status	-- SSE or poll for processing status
POST	/courses/generate	-- trigger course generation
GET	/courses/{id}	-- full course tree
GET	/courses	-- user's courses (dashboard)
POST	/progress/lessons/{id}/complete	
GET	/progress/courses/{id}	
POST	/chat	-- SSE streaming chat response
GET	/chat/conversations/{course_id}	
POST	/quizzes/generate	
POST	/quizzes/{id}/submit	
GET	/quizzes/{id}/attempts	
GET	/search?q=	
GET	/history	
GET	/dashboard	

16. Build Order (Phased Plan)

Phase 1 — Foundation (Days 1–2) Auth wired end-to-end (Next.js Auth.js FastAPI JWT verification, Google + email/password + verification + reset flows); Neon schema and migrations (Alembic); basic dashboard shell.

Phase 2 — Ingestion Pipeline (Days 3–4) Upload → object storage → background worker → text extraction → chunking → embeddings → pgvector storage.

Phase 3 — Course Generation (Days 5–7) Outline generation prompt, lesson generation prompts, persistence to the course schema, course viewer UI, SSE progress streaming.

Phase 4 — Progress and Dashboard (Day 8) Completion tracking, resume-learning logic, dashboard aggregation endpoint and UI.

Phase 5 — Chatbot / RAG (Days 9–10) Vector search integration, streaming chat responses, chapter-summarize and quiz-me intents.

Phase 6 — Quizzes (Day 11) Quiz generation, grading logic (exact match + LLM semantic grading), attempts UI.

Phase 7 — Search and History (Day 12) Full-text search, semantic search, history timeline aggregation.

Phase 8 — Bonus Features (Days 13–15) Select 3–5 highest-value bonus features: flashcards, mind maps, streaks, certificates, and dark mode are generally the best effort-to-impact ratio.

Phase 9 — Polish and Deployment (Days 16–17) Error handling, loading states, full responsive pass, rate limiting; deploy frontend to Vercel, backend and worker to Render/Railway/Fly.io, with Neon already hosted; write API documentation (FastAPI's auto-generated OpenAPI/Swagger docs), README, and environment setup instructions.

17. Notes on Groq and Free-Tier Constraints

- Groq’s free tier enforces rate limits (requests per minute and tokens per minute, varying by model) — the background worker should **queue and throttle** LLM calls rather than firing them all at once during course generation, to avoid 429 errors on larger PDFs.
- Choose a Groq model that balances output quality and speed for structured JSON generation (their Llama 3.x family is a solid default); confirm current model names and limits against Groq’s documentation, as these change over time.
- Cache aggressively: avoid regenerating a course if the same PDF is re-uploaded, by hashing the file and checking for an existing **document** row with a matching hash.

18. What Will Score Well in Evaluation

- **Properly implemented RAG** — grounded lessons and grounded chatbot answers, rather than simply calling the LLM with the entire PDF as context — is likely the single biggest differentiator.
- **Streaming UX** — Groq’s speed advantage is wasted if the UI waits for a full response before showing anything.
- **A clean async backend** with a real background job queue, instead of blocking the HTTP request during PDF processing.
- **Solid error handling** for corrupted PDFs, oversized files, LLM timeouts and rate limits, and partial course-generation failures with retry logic.
- **Thorough documentation** — a clear README, an architecture diagram, and generated OpenAPI docs tend to stand out more than expected during evaluation.